AI Automation with n8n

Lesson 4: Integrating OpenAI GPT

Integrating OpenAI GPT - Study Notes

Key Concepts

1. **OpenAI Node** – The pre built component that lets your workflow communicate with the OpenAI API.

2. **Prompt Construction** – How to format the text you send so the model understands the task.

3. **API Request & Response Cycle** – Sending a request, receiving JSON, and extracting the generated text.

4. **Error Handling & Rate Limits** – Detecting failures, retrying, and staying within usage limits.

5. **Security & Secrets Management** – Storing the API key safely (environment variables, secret vaults, etc.).

Summary

Integrating OpenAI's GPT models into an application is essentially a three step process: add the OpenAI node (or SDK), send a properly formatted prompt, and handle the response. The node abstracts the HTTP request, letting you focus on the content of the prompt and what you do with the returned text.

When you configure the node, you provide your API key (kept in a secure secret store) and select the model (e.g., `gpt-4o-mini`). The prompt can be a simple string or a structured chat array, depending on whether you need a single completion or a multi turn conversation. After the node runs, the response arrives as JSON containing fields such as `choices[0].message.content`. Extract that field, pass it to downstream nodes, or display it to the user.

Robust integrations also include error handling: checking for HTTP status codes, catching rate limit (`429`) responses, and implementing exponential back off retries. Keeping the API key out of source code, using environment variables or a secret manager, protects your account and complies with best practice security.

Important Points

• **Node Setup**

- Drag the **OpenAI** node into your workflow.

- Set **Authentication** !' **API Key** (store in a secret).

- Choose **Model** (e.g., `gpt-4o`, `gpt-3.5-turbo`).

• **Prompt Formats**

- **Simple Completion**: `{"prompt": "Explain photosynthesis in two sentences."}`

- **Chat Completion**: `{"messages": [{"role":"system","content":"You are a helpful tutor."}, {"role":"user","content":"What is photosynthesis?"}]}`

• **Sending the Request**

- Most nodes expose an **Input** field where you map variables (e.g., `{{payload.text}}`).
- Optional parameters: `temperature`, `max_tokens`, `top_p`, `frequency_penalty`.
    • **Handling the Response**
- Response JSON structure:
  ```json
  {
    "id": "...",
    "object": "chat.completion",
    "choices": [
      {
        "message": {"role":"assistant","content":"..."},
        "finish_reason":"stop",
        "index":0
      }
    ],
    "usage": {"prompt_tokens":..., "completion_tokens":..., "total_tokens":...}
  }
  ```
- Extract the assistant's text: `{{response.choices[0].message.content}}`.
    • **Error Management**
- 401 /!' /Invalid API key.
- 429 /!' /Rate limit exceeded – wait (e.g., 2 /s, then 4 /s, then 8 /s).
- Network errors – retry up to 3 times with back off.
    • **Security**
- Never hard code the key.
- Use environment variables (`OPENAI_API_KEY`) or a secret manager.
- Rotate keys regularly.

Examples

Example 1 – Simple Text Completion

Goal: Generate a tagline for a coffee shop.

```mermaid
flowchart TD
    A[Start] --> B[Set variable: shopName = "Brew Haven"]
    B --> C[OpenAI Node]
    C --> D[Extract content]
    D --> E[Display result]
```

Configuration (OpenAI node):

Field	Value

| Model | gpt-3.5-turbo |

| Prompt (JSON) | `{ "prompt": "Create a catchy tagline for a coffee shop called {{shopName}}.", "max_tokens": 20 }` |

| Temperature | 0.7 |

| Output mapping | `tagline = {{choices[0].text}}` |

Result: `"Brew Haven – Where Every Sip Tells a Story"`

Explanation: The prompt is concise, the model returns a short string, and we map that string to `tagline` for later use.

Example 2 – Multi Turn Chat for Customer Support

Goal: Simulate a support bot that answers a user's question about order status.

```mermaid
sequenceDiagram
    participant User
    participant Flow
    participant OpenAI
    User->>Flow: "Where is my order #12345?"
    Flow->>OpenAI: System + User messages
    OpenAI-->>Flow: Assistant reply
    Flow->>User: "Your order is in transit and will arrive tomorrow."
```

Node Input (Chat format):

```json
{
  "messages": [
    {"role":"system","content":"You are a friendly e  commerce support assistant. Keep answers brief and helpful."},
    {"role":"user","content":"{{incomingMessage}}"}
  ],
  "temperature": 0.5,
  "max_tokens": 60
}
```

Response Mapping:

- `assistantReply = {{choices[0].message.content}}`

Explanation: By providing a system message we set the assistant's tone. The user's incoming text is injected via `{{incomingMessage}}`. The bot's reply is captured and sent back to the user.

Quick Review

1. **What is the purpose of the OpenAI node in a workflow?**
2. **Name two ways to format a prompt for the API.**
3. **Which JSON field contains the generated text in a chat completion response?**

4. **What should you do when you receive a 429 status code?**

5. **Why must the API key be stored in a secret manager rather than hard coded?**

Further Reading

- **OpenAI API Reference** – Deep dive into all parameters (`logit_bias`, `response_format`, etc.).
- **Rate Limiting & Quotas** – Understanding token limits and best practices for batching requests.
- **Prompt Engineering** – Techniques for improving model reliability and creativity.
- **Secure Secrets Management** – Using Vault, AWS Secrets Manager, or environment variable best practices.
- **Streaming Responses** – Using the `stream` flag for real time token delivery.

End of study notes.